湘潭市政务短信服务平台

对接文档

2024 年 11 月 8 日

文档修订记录

版本号	变化状态	简要说明	日期	变更人	批准日期	批准人
V1.0	C	创建	2024118	彭翠翠		

变化状态：C = 创立，A = 增加，M = 修改，D = 删除

目录

湘潭市政务短信服务平台 1
对接文档 1
文档修订记录 2
第一章 说明 4
1.1. 接入步骤 4
1.2. 注意事项 4
第二章 开通测试账号 4
第三章 创建对接应用 4
3.1. 创建应用 5
3.2. 应用提交审核 5
3.3. 查看应用密钥 6
第四章 应用短信发送 6
4.1 服务器 api 地址 6
4.2 接口调用说明 6
4.3 发送接口 7
4.4 通知接口 16

第一章 说明

1.1. 接入步骤

1）开通短信平台登陆账号 → 创建应用 →获取短信对接 ak 以及 sk → 短信发送接口对接 → 回执回复回调接口对接

1.2. 注意事项

1）应用系统的用户账号，必须从统一身份认证平台同步而来，不能自行新建与统一身份认证平台无关的用户账号。

2）需先提交短信平台申请表以及开通 10.129.17.93：443 的网络策略

3）为了对接顺畅，请提前对网络通断进行测试。命令如下：

telnet 10.129.17.93 443

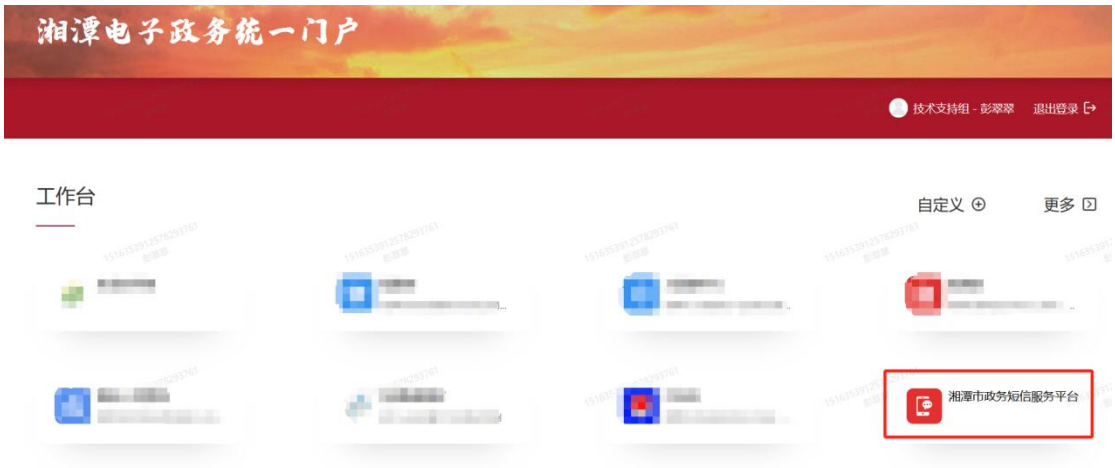
telnet sms.xtdzzw.cn 443

第二章 开通测试账号

请联系该应用的使用单位进行开通（可直接使用该单位的用户账号），开通测试时请提供以下信息：姓名、手机号、名称、项目名称、使用单位、是否需要签名以及签名内容。

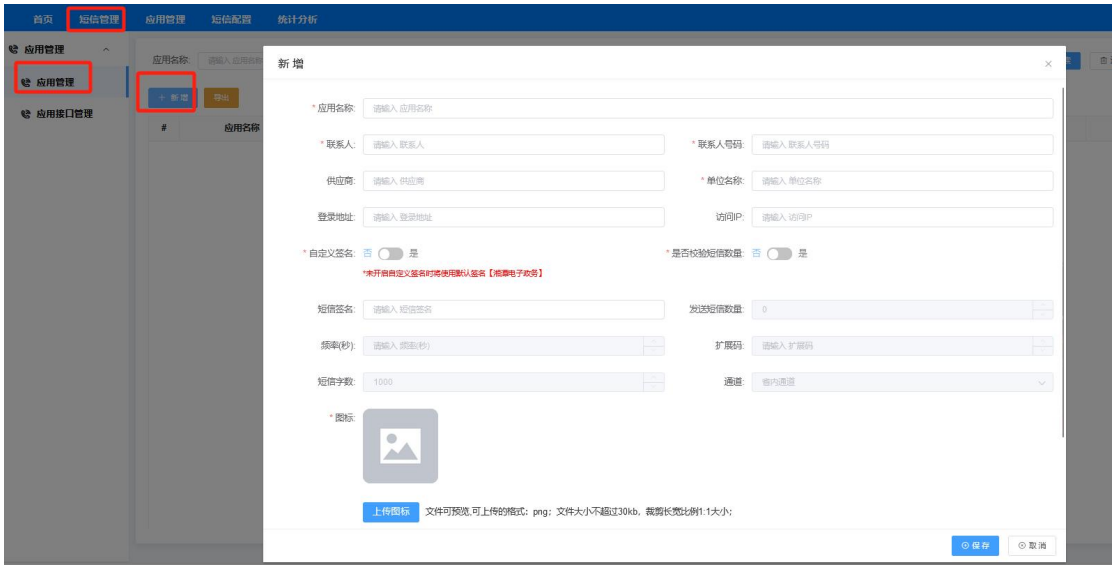
第三章 创建对接应用

开通账号以后，可以登录统一身份认证平台，统一身份认证平台地址：https://www.xtdzzw.cn/（该统一认证目前只在政务网内部能通，互联网是不能进行访问和对接）。



3.1. 创建应用

登陆到短信平台之后，通过短信管理 -> 应用管理 -> 新增 创建应用



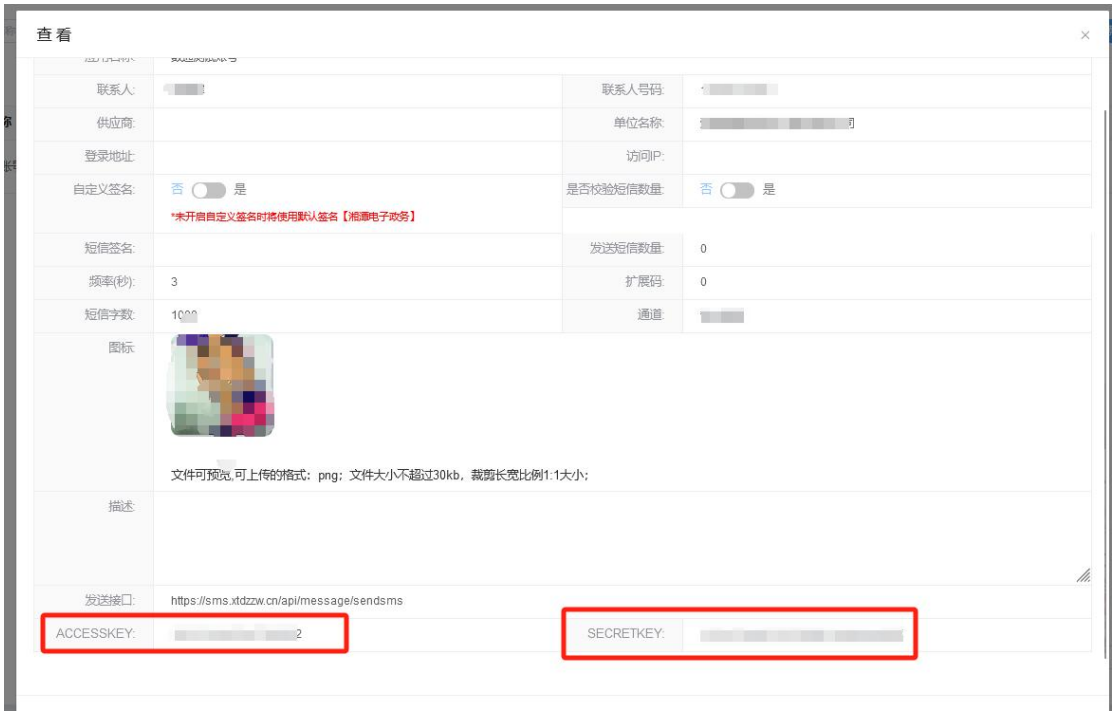
进入到创建新应用界面，请填写好应用名称、联系人、联系人电话，单位名称，自定义签名等以及上传应用 logo。

3.2. 应用提交审核

应用创建完成后，可以在应用申请处修改该应用的相关信息，确认应用信息无误后，可以点击提交按钮，等待管理员进行审核。

3.3. 查看应用密钥

应用已审核通过，即可在应用列表中查看该应用，在应用详情下方即可查看密钥的相关信息。



第四章 应用短信发送

4.1 服务器 api 地址

前缀 https://sms.xtdzzw.cn/api

完整的 API 地址为：前缀+具体接口路径

比如，单条下发接口为：

```
https://sms.xtdzzw.cn/api/ + message/smsSingle
```

4.2 接口调用说明

1）签名说明

参数按 ascii 码升序排序拼接成 queryParam 形式,最后拼接 key=xxxx(apiKey 秘钥),使用 MD5 加密,并转大写! 见具体接口参数说明

如:原参数,

```
{
"accessKey":"100001",
"timestamp":1700811931489,
"sign":"",
"desAddr":"136xxxxxxxx",
"msgContent":"【示例】验证码:1234"
}
```

排序后：accessKey=100001&desAddr=139xxxxxxxx&msgContent=【示例】验证码:1234&timestamp=1700811931489&key=88888888

md5 加密:22f7b91bd1cbad50965657b649941e4b

转大写后:22F7B91BD1CBAD50965657B649941E4B

4.3 发送接口

1.单内容批量发送

多个手机号码，单个内容，无 guId

Request

method：post

Url:/message/sendsms

Headers:application/json;charset=utf-8

Body:

```
{
"accessKey":"",
"timestamp":,
"sign":"",
"desAddr":"",
"msgContent":"",
```

```
}
```

参数说明

参数名称	参数类型	参数说明	实例值	是否必填
accessKey	字符串	分配给客户的账号		是
timestamp	数字类型	时间戳		是
sign	字符串	签名数据		是
desAddr	字符串	手机号码	多个用，隔开 13xx,15xx	是
msgContent	字符串	发送内容		是

Response

```
{

"code": "200", //状态码 200 返回成功

"success": true,

"msg": "OK",

"data":[

  {

  "phone":"139xxxxxx"

 "guId":"xxxxxxxxxx"

  }

 ]

}
```

代码示例：

```
public class SendMessVo {

 /** * 签名 */
```

```
private String sign;

/** * 时间戳 */

private Long timestamp;

 /** * ak */

private String accessKey;

 /** * 手机号码 */

private String desAddr;

 /** * 发送内容 */

private String msgContent;

}

public String test() throws Exception {

SendMessVo sendMessVo = new SendMessVo(); sendMessVo.setAccessKey("akxxxxx");

sendMessVo.setDesAddr("139xxxxxxx,189xxxxxxxx");//可多个手机号码 sendMessVo.setTimestamp(System.currentTimeMillis());

sendMessVo.setMsgContent("【示例】验证码:1234");

Map<String,Object> map = BeanUtil.beanToMap(sendMessVo); map.remove("sign");

//排序

String[] sortedKeys = map.keySet().toArray(new String[]{}); Arrays.sort(sortedKeys); StringBuilder s2 = new

StringBuilder();

for (String key : sortedKeys) {

s2.append(key).append("=").append(map.get(key)).append("&");

}

s2.append("key=skxxxx");

String md5 = Diges tUtils.md5DigestAsHex(s2.toString().getBytes()); sendMessVo.setSign(md5.toUpperCase());

HttpClient httpClient = HttpClients.createDefault();

String url = "ip:prot/message/sendsms";

// 指定的 URL

HttpPost request = new HttpPost(url);

// 设置请求体

StringEntity requestBody = new StringEntity(JSON.toJSONString(sendMessVo), Charset.forName("UTF-8"));

requestBody.setContentType("application/json"); request.setEntity(requestBody);

// 发送请求并获取响应
```

```
HttpResponse response = httpClient.execute(request);

// 处理响应

HttpEntity responseEntity = response.getEntity();

String responseString = EntityUtils.toString(responseEntity);

return responseString;

}
```

2.点对点批量发送

一个号码对应一个内容对应一个 guId

Request

method：post

Url:/message/smsBatch

Headers:application/json;charset=utf-8

Body:

```
{

"accessKey":"",

"timestamp":,

"sign":"",

sendData:[

{

"desAddr":"",

"msgContent":"",

"guId":"xxxxxxxx"

}

]

}
```

参数说明

参数名称	参数类型	参数说明	实例值	是否必填

accessKey	字符串	分配给客户的账号		是
timestamp	数字类型	时间戳		是
sign	字符串	签名数据		是
sendData	数组	短信内容（不参与签名）		是
desAddr	字符串	手机号码	153xxxxxxxx	是
msgContent	字符串	发送内容		是
guId	字符串	guId(唯一)		是

Response

```
{

"code": "200", //状态码 200 返回成功

"success": true,

"msg": "OK",

"data":[

  {

   "phone":"139xxxxxx",

   "guId":"xxxxxxxxxxx"

  },

  {

   "phone":"189xxxxxx",

   "guId":"xxxxxxxxxxx"

  }

 ]
```

```
}
```

代码示例：

```
@Data

public class SendMessVo {

            /** * 签名 */

private String sign;

            /** * 时间戳 */

private Long timestamp;

/** * ak */

private String accessKey;

/** * 发送内容 */

 List<SendDataVo> sendData;

}



@Data public class SendDataVo {

/** * 手机号码 只能单个手机号码 */

private String desAddr;

/** * guId */

private String guId;

 /** * 发送内容 */

 private String msgContent;

}



public String testSend(MessageContentVo contentVo) throws Exception {

SendMessVo sendMessVo = new SendMessVo();

String [] phones = contentVo.getPhone().split(",");

            sendMessVo.setAccessKey(smsSendConfig.getSmsAccessKey());       List<SendDataVo> dataVos = new
ArrayList<>();

for(int i = 0 ; i < phones.length;i++){
```

```
SendDataVo sendDataVo = new SendDataVo();           sendDataVo.setDesAddr(contentVo.getPhone());
sendDataVo.setMsgContent(contentVo.getSmsContent()); sendDataVo.setGuId("guidxxxxxx"); dataVos.add(sendDataVo); }
sendMessVo.setTimestamp(System.currentTimeMillis()); Map<String,Object> map = BeanUtil.beanToMap(sendMessVo);
map.remove("sign");

//排序

String[] sortedKeys = map.keySet().toArray(new String[]{}); Arrays.sort(sortedKeys);     StringBuilder s2 = new
StringBuilder(); for (String key : sortedKeys)     {

s2.append(key).append("=").append(map.get(key)).append("&");

}

s2.append("key="+smsSendConfig.getSmsSecretKey()); String md5 =
DigestUtils.md5DigestAsHex(s2.toString().getBytes());
sendMessVo.setSign(md5.toUpperCase());
sendMessVo.setSendData(dataVos);

HttpClient httpClient = HttpClients.createDefault();

String url =              smsSendConfig.getSmsSendUrl();

 // 指定的 URL

HttpPost request = new    HttpPost(url);

// 设置请求体

StringEntity requestBody = new        StringEntity(JSON.toJSONString(sendMessVo),Charset.forName("UTF-8
             "));

requestBody.setContentType("application/json");     request.setEntity(requestBody);

// 发送请求并获取响应

HttpResponse              response = httpClient.execute(request);

// 处理响应

HttpEntity   responseEntity = response.getEntity();

String responseString =   EntityUtils.toString(responseEntity); return responseString;

}
```

3.单条发送

一个手机号码对应一个内容

Request

method：post

Url:/message/smsSingle

Headers:application/json;charset=utf-8

Body:

```
{
"accessKey":"",
"timestamp":,
"sign":"",
"desAddr":"",
"msgContent":"",
"guId":""
}
```

参数说明

参数名称	参数类型	参数说明	实例值	是否必填
accessKey	字符串	分配给客户的账号		是
timestamp	数字类型	时间戳		是
sign	字符串	签名数据		是
desAddr	字符串	手机号码	139xxxxxxxx	是
msgContent	字符串	发送内容		是
guId	字符串	guId		否

Response

```
{
"code": "200", //状态码 200 返回成功
"success": true,
"msg": "OK",
"data":[
   {
    "phone":"139xxxxxx",
    "guId":"xxxxxxxxxxx"
   }
  ]
}
```

代码示例：

```
public class SendMessVo {
/** * 签名 */
private String sign;
/** * 时间戳 */
private Long timestamp;
/** * ak */
private String accessKey;
/** * 手机号码，只能单个手机号码 */
private String desAddr;
/** * 发送内容 */
private String msgContent;
/** * guId 用户自定义唯一字段 */
private String guId;
}
public String test() throws Exception {
SendMessVo sendMessVo = new SendMessVo(); sendMessVo.setAccessKey("akxxxxx"); sendMessVo.setDesAddr("139xxxxxxxx");
sendMessVo.setTimestamp(System.currentTimeMillis()); sendMessVo.setMsgContent("【示例】验证码:1234");
```

```
sendMessVo.setGuId("guidxxxxxx");

Map<String,Object> map = BeanUtil.beanToMap(sendMessVo); map.remove("sign");

//排序

String[] sortedKeys = map.keySet().toArray(new String[]{}); Arrays.sort(sortedKeys);

StringBuilder s2 = new StringBuilder();

for (String key : sortedKeys) {

s2.append(key).append("=").append(map.get(key)).append("&");

}

s2.append("key=skxxxx");

String md5 = DigestUtils.md5DigestAsHex(s2.toString().getBytes()); sendMessVo.setSign(md5.toUpperCase());

HttpClient httpClient = HttpClients.createDefault();

String url = "ip:prot/message/sendsms";

// 指定的 URL

HttpPost request = new HttpPost(url);

// 设置请求体

StringEntity requestBody = new StringEntity(JSON.toJSONString(sendMessVo), Charset.forName("UTF-8"));

requestBody.setContentType("application/json"); request.setEntity(requestBody);

// 发送请求并获取响应

HttpResponse response = httpClient.execute(request);

// 处理响应

HttpEntity responseEntity = response.getEntity();

String responseString = EntityUtils.toString(responseEntity);

return responseString;

}
```

4.4 通知接口

1. 回执报告通知

Request

method：post

Url：客户提供

Headers:application/json;charset=utf-8

Body:

```
{
"receiptCode":"",
"receiptDesc":,
"guId":""
}
```

参数说明

参数名称	参数类型	参数说明	实例值
receiptCode	字符串	回执状态码	
receiptDesc	字符串	回执描述	
guId	字符串	消息 id	

2.回复通知

Request

method：post

Url:客户提供

Headers:application/json;charset=utf-8

Body:

```
{
"replyTime":"",
"phone":,
"guId":"",
```

```
"content":"",

}
```

参数说明

参数名称	参数类型	参数说明	实例值
replyTime	字符串	回复时间	
phone	字符串	手机号码	
guId	字符串	消息 id	
content	字符串	回复内容	